

APUNTES PARA EXAMEN

DBML · DBDiagram · RunSQL · QA/QC/QE · UI/UX · PM/PO · Scrum · Tipos de Pruebas

1 · DBML — Database Markup Language

¿Qué es?	Lenguaje de marcado de código abierto, simple y legible, diseñado para definir esquemas de bases de datos mediante código.
Filosofía	"Documentación como código". Fácil de compartir y versionar (Git).
Utilidad	Facilita el diseño visual (diagramas Entidad-Relación) antes de pasar a la programación real en SQL.
Flujo típico	1) Diseñar el esquema en DBML → 2) Generar el diagrama E-R → 3) Exportar SQL → 4) Implementar en la base de datos.

2 · DBDiagram.io

¿Qué es?	IDE web especializado que interpreta código DBML y dibuja el diagrama E-R en tiempo real.
Tiempo real	Escribes código a la izquierda → el diagrama se genera a la derecha.
Importar SQL	Permite subir archivos .sql y convertirlos automáticamente a diagramas DBML.
Exportar SQL	Genera el código SQL listo para usar en PostgreSQL, MySQL o SQL Server.
Colaboración	Genera enlaces públicos para trabajar en equipo de forma compartida.

3 · RunSQL.com

¿Qué es?	Entorno de ejecución y testeo web que permite validar bases de datos mediante SQL real sin configurar servidores locales.
Interfaz (izq.)	Se define el esquema y se visualiza el contenido de las tablas.
Interfaz (der.)	Se redactan las consultas y se comprueban los resultados obtenidos.
Flujo profesional	1) Introducir datos de prueba (datos dummy) en el esquema. 2) Testear la lógica de las relaciones. 3) Validar sentencias antes de producción.
Clave	Sin datos dummy la herramienta no es eficaz: las consultas no devolverán resultados reales que permitan detectar errores.

4 · QA · QC · QE — Calidad del Software

	QA — Quality Assurance	QC — Quality Control	QE — Quality Engineering
Significado	Aseguramiento de la Calidad	Control de la Calidad	Ingeniería de la Calidad
Tipo	Proactivo / orientado al proceso	Reactivo / orientado al producto	Integral / transversal
Objetivo	Prevenir defectos estableciendo estándares, metodologías y procedimientos sistemáticos.	Identificar y corregir defectos en el producto ya construido.	Incorporar la calidad en todo el ciclo de vida de forma continua y escalable.
Herramientas	Auditorías, formación, revisión de requisitos, estándares de proceso.	Testing, inspección de código, seguimiento de bugs, reportes.	Automatización de pruebas, CI/CD, pipelines, análisis de cobertura.
Rol típico	QA Analyst / QA Manager	QC Tester / Manual Tester	SDET / Automation Engineer
¿Cuándo actúa?	Durante todo el SDLC.	Al final de cada fase o sprint.	Transversal: desde planificación hasta mantenimiento.
Analogía	El arquitecto que diseña normas para evitar problemas.	El inspector que revisa el edificio y encuentra grietas.	El ingeniero que crea las máquinas para construir mejor.

■ **IMPORTANTE:** En resumen: QA previene errores mejorando el proceso · QC detecta errores probando el producto · QE automatiza e integra la calidad en todo el ciclo de desarrollo.

5 · UI / UX — Interfaz y Experiencia de Usuario

Característica	UI — User Interface	UX — User Experience
Enfoque	El producto (el dispositivo). Look & feel.	El usuario (la persona). Feeling / reacción.
¿Qué trata?	Herramientas, botones, colores y tipografía.	Usabilidad, análisis, lógica y satisfacción.
Objetivo	Crear una interfaz estéticamente atractiva y coherente con la marca.	Que el usuario encuentre solución a su problema de forma fácil, lógica y satisfactoria.
Alcance	Es una parte de la experiencia de usuario.	Es el todo: el conjunto de factores de la interacción.
¿Cuándo?	Actúa tras definir estructura y flujos.	Actúa desde la fase de investigación inicial.
Impacto	Primera impresión y percepción de calidad.	Retención y satisfacción del usuario a largo plazo.
Entregable	Pantallas y componentes visuales.	Wireframes, prototipos y flujos de usuario.

■ **IMPORTANTE:** Una interfaz hermosa (excelente UI) no sirve de nada si el sitio es difícil de usar (mal UX), y viceversa. Ambas disciplinas se necesitan mutuamente.

6 · Product Manager (PM) vs Product Owner (PO)

Aspecto	Product Manager (PM)	Product Owner (PO)
Rol principal	«Dueño del producto» ante el mercado.	Rol específico dentro del marco Scrum.

Aspecto	Product Manager (PM)	Product Owner (PO)
Visión	Largo plazo / orientada al exterior (mercado y clientes).	Corto/medio plazo / orientada al interior (equipo dev).
Estrategia	Define la estrategia del producto y hacia dónde debe ir.	Traduce la visión del PM en tareas concretas (Historias de Usuario).
Negocio	Garantiza que el producto sea viable económicamente y cumpla los objetivos de la empresa.	—
Backlog	—	Responsable de gestionar y priorizar el Product Backlog para cada Sprint.
Descubrimiento	Habla con los clientes para decidir qué funcionalidades construir.	Se asegura de que el trabajo técnico sea exactamente lo necesario.

■ **IMPORTANTE:** El PM define QUÉ construir; el PO asegura que se construya CORRECTAMENTE. El PO actúa como enlace con el cliente cuando el PM no está presente.

7 · SCRUM — Marco de Trabajo Ágil

Framework ágil y ligero para el desarrollo iterativo e incremental. El producto se construye en ciclos cortos llamados **Sprints**.

Roles

Rol	Función principal	Responsabilidad clave
Product Owner	Enlace con el cliente. Valida y certifica las peticiones.	Crea y gestiona el Product Backlog.
Scrum Master	Organizador, moderador y facilitador. No da órdenes.	Asegura que se cumplan las reglas de Scrum. Lee el Product Backlog y ayuda a definir el Sprint Backlog.
Developer Team	Ejecuta el desarrollo de los Sprints.	Equipo multifuncional (analistas, diseñadores, programadores, testers). Autoorganizado.

Artefactos (Documentos)

Product Backlog	Lista completa de funcionalidades y requisitos del cliente. Es el inventario de todo lo que el producto podría necesitar. No cambia de sprint en sprint.
Sprint Backlog	Subconjunto de tareas extraídas del Product Backlog que el equipo se compromete a completar en el Sprint actual.
Incremento	Versión funcional y potencialmente entregable del producto al final de cada Sprint.

Ceremonias (Fases del Sprint)

Ceremonia	¿Cuándo?	Objetivo	Detalle
Sprint Planning	Inicio del Sprint	Planificar el Sprint.	El equipo y el Scrum Master planifican las soluciones. Los analistas recogen requisitos del Sprint Backlog.
Daily Meeting	Cada día (15 min)	Sincronización diaria.	Planificar las tareas del día y analizar el progreso conseguido.
Sprint Review	Fin del Sprint	Presentar el incremento al cliente.	Se explica qué tareas fueron completadas y cuáles no. Se valida el cumplimiento y se planifican nuevas prioridades.
Sprint Retrospective	Tras la Review	Mejora continua del proceso interno.	Reunión interna del equipo. Se analizan errores y aciertos. Se dejan por escrito compromisos y mejoras concretas para el próximo Sprint.

8 · Tipos de Pruebas

Visión General

Tipo	También llamada	Quién la realiza	Enfoque	Objetivo
Caja Blanca	Estructural / Clear-box	Tester con acceso al código fuente	El CÓMO se hace (código interno)	Que cada línea de código se ejecute al menos una vez.

Tipo	También llamada	Quién la realiza	Enfoque	Objetivo
Caja Negra	Funcional / Behavioural	Tester sin acceso al código	El QUÉ hace (funcionalidad visible)	Que la salida (output) sea correcta para cada entrada (input).
Regresivas	Regression Testing	QA / Tester	Estabilidad tras cambios	Asegurar que nada que ya funcionaba se ha roto con una actualización.

Técnicas de Caja Blanca

Técnica	Objetivo	Cómo se aplica
De cubrimiento	Que cada línea de código del programa se ejecute al menos una vez.	Se mide el porcentaje de cobertura del código; el objetivo es el 100%.
De condiciones	Probar todos los resultados posibles (verdadero/falso) de cada sentencia condicional.	Se diseñan casos para que cada IF, WHILE y SWITCH tome valor true y false al menos una vez.
De bucles	Comprobar los límites de las repeticiones para evitar bucles infinitos o errores de rango.	Se prueba el valor máximo, máximo+1 y máximo-1 del bucle.

Técnicas de Caja Negra

Técnica	Objetivo	Ejemplo práctico
Clases de equivalencia	Clasificar los datos de entrada en válidos e inválidos.	Comprobar que una contraseña cumple o no los requisitos de longitud. Se prueba un representante de cada clase, no todos los valores.
Valores límite	Probar datos extremos o irreales en los bordes de cada clase de equivalencia.	Introducir una edad de 150 años vs. 25 años para ver cómo reacciona el sistema. Si un campo acepta 1–100: se prueban 0, 1, 100 y 101.
Prueba de interfaz (GUI)	Evaluar la interfaz gráfica y su usabilidad.	Verificar atajos de teclado, ventanas emergentes y la navegación general. Si la caja negra detecta un fallo, se recurre a caja blanca para arreglarlo.

■ **IMPORTANTE:** Relación Caja Negra ↔ Caja Blanca: la Caja Negra detecta el fallo; la Caja Blanca localiza y repara el error en el código fuente.